Software Requirements and Design Document

For

Fantasy Football

Version 1.0

Authors:
Miles G
Tanner S
Amadou L
Edward P

1. Overview (5 points)

Ultimate Fantasy Football is a web app built to make fantasy football more hands-on and strategic throughout the week, rather than just checking scores on Sundays. Developed with React/TypeScript on the front end and a Python, flask, and supabase backend, the platform puts players in the role of a head coach. Users can pick specific coach traits at the start of the season (like a run-heavy scheme for running back multipliers), and run daily trivia minigames to prep for matchups. To keep things interactive, teams also manage their home stadium atmosphere and training facilities to keep players fresh and prevent simulated weekly atrophy.

From a technical standpoint, the biggest challenge is pulling near-live game stats while staying under Tank01's free tier of 1,000 API calls a month. Because coach boosts are calculated locally in our own database with zero external calls, practically our entire API budget is saved for a scheduled caching service that pulls live box scores every 5 to 10 minutes only while games are being played.

2. Functional Requirements (10 points)

Accounts and Roles
1. The system will allow a visitor to register an account with a unique email address, a unique username, and a password. **(High)**

2. The system will allow a registered user to log in and log out. **(High)**

3. The system will support two roles: User and League Manager. **(High)**

4. The system will grant League Managers access to all User features plus the League Management View. **(High)**

5. The system will automatically assign the League Manager role to the user who creates a league, for that league only. **(High)**

6. The system will restrict League Management functions so that Users who are not the manager of a league cannot access them. **(High)**

2.2 League Management (League Manager)
7. The system will allow a League Manager to create a league with a name and a maximum number of teams (minimum 4, maximum 16). **(High)**

8. The system shall allow a League Manager to invite users to the league through a shareable invite code or link. **(High)**

9. The system shall prevent users from joining a league that has reached its maximum team count. **(High)**

2.3 Roster Management
10. The system shall allow a User to view their roster, including each player's position, real-life team, and current status. **(High)**

11. The system shall allow a User to set their starting lineup (e.g., 1 QB, 2 RB, 2 WR, 1 TE, 1 FLEX, 1 K, 1 DEF) and place the remaining players on the bench. **(High)**

12. The system shall lock a player in a User's lineup once that player's real-life game begins. **(High)**

13. The system shall reject a lineup that violates the required position slots and tell the User which slot is invalid. **(High)**

14. The system shall allow a User to add a free agent to their roster and drop a rostered player in the same transaction. **(High)**

15. The system shall display a Free Agency View listing all unrostered players, filterable by position and searchable by name. **(High)**

16. The system shall prevent a User from exceeding the maximum roster size defined for the league. **(High)**

2.4 Scoring, Scoreboard, and History

17. The system shall calculate each player's fantasy points from real-life statistics using the league's scoring rules. **(High)**

18. The system shall apply active coach-trait multipliers, and penalties to a player's base fantasy points to produce the final score. **(High)**

19. The system shall display the breakdown of each player's final score (base points, coach multiplier, chemistry bonus, atrophy penalty, minigame bonus). **(High)**

20. The system shall display a Weekly Scoreboard showing every matchup in the league with live-updating scores during games. **(High)**

21. The system shall determine each matchup's winner at the end of the week and update league standings (wins, losses, ties, total points). **(High)**

22. The system shall allow a User to view any previous week's lineup, player scores, and matchup result. **(Medium)**

23. The system shall display an Upcoming Schedule of the User's future fantasy matchups and relevant real-life games. **(Medium)**

2.5 Coach Traits

24. The system shall require each User to select one coach trait at the start of the season (e.g., Run-Focused, Pass-Focused, Defensive-Minded). **(High)**

25. The system shall apply the selected trait's multiplier to the fantasy points of the affected player positions (e.g., Run-Focused increases RB points by a fixed percentage). **(High)**

26. The system shall not allow a User to change their coach trait after the season begins. **(Medium)**

27. The system shall cap the total combined effect of multipliers and bonuses so that base real-life performance remains the dominant factor in a player's score. **(High)** Rationale: the proposal's risk section states that big real-life performances should not be minimized by the added features. A cap is a concrete way to enforce that.

2.7 Daily Minigame

33. The system shall present each User with one trivia minigame per calendar day. **(Medium)**

34. The system shall record a User's minigame result and limit each User to one attempt per day. **(Medium)**

35. The system shall convert a User's minigame results during a week into a bonus multiplier applied to that week's score. **(Medium)**

36. The system shall reset the accumulated minigame bonus at the start of each new fantasy week. **(Medium)**

2.8 Home Stadium and Training Facility

37. The system shall provide each User with a Home Stadium page displaying stadium condition, fan attendance, and crowd noise level. **(Low)**

39. The system shall change fan attendance and crowd noise based on stadium condition and team performance. **(Low)**

40. The system shall apply a home-field bonus to a User's score in weeks when they are the home team, scaled by crowd noise level. **(Low)**

41. The system shall allow a User to upgrade their Training Facility through defined levels using in-game currency. **(Low)**

42. The system shall award in-game currency to Users based on weekly results and minigame performance. **(Low)**

2.9 Data and Statistics
43. The system shall retrieve player, schedule, and live game statistics from the Tank01 NFL API. **(High)**

44. The system shall update live statistics during games at least every 5 minutes, matching the API's data freshness. **(High)**

45. The system shall serve statistics to Users from a cache rather than calling the external API for each User request. **(High)**

46. The system shall display the time of the last successful data update on the scoreboard. **(Medium)**

47. The system shall notify Users that data may be delayed if the external API is unavailable, and continue to show the last cached data. **(Medium)**

NON-FUNCTIONAL REQUIREMENTS

3. Non-functional Requirements (10 points)

1. [Performance] The system shall load any page in under 3 seconds under normal conditions for cached content.

2. [Performance] The system shall reflect a lineup or roster change in the interface within 2 seconds of submission.

3. [Performance] The system shall support at least 100 concurrent users without noticeable degradation. Rationale: appropriate scale for class demonstration and testing.

4. [API Usage] The system shall keep external API usage within the Tank01 free tier limit of 1,000 calls per month during development and demonstration, by using caching and scheduled refreshes. Rationale: this is the main risk identified in the proposal.

5. [API Usage] The system shall poll the external API only while at least one NFL game is in progress, and use a much lower refresh frequency otherwise.

6. [Security] The system shall store passwords only as salted hashes, never in plaintext.

7. [Security] The system shall transmit all data over HTTPS.

8. [Security] The system shall enforce role-based authorization on the server side for every request, not just in the interface.

9. [Security] The system shall protect against common web vulnerabilities including SQL injection, cross-site scripting, and cross-site request forgery.

10. [Security] The system shall keep API keys out of client-side code and the source repository.

11. [Reliability] The system shall remain available with at least 99% uptime during scheduled NFL game windows.

12. [Reliability] The system shall not lose committed user data (rosters, lineups, scores) in the event of a server restart.

13. [Accuracy] The system shall calculate scores deterministically, so the same inputs always produce the same final score.

14. [Accuracy] The system shall be able to recompute any week's scores from stored statistics, to correct for stat corrections issued after games.

15. [Usability] The system shall allow a new user to register, join a league, and set a lineup without external instructions.

16. [Usability] The system shall be fully usable on the current versions of Chrome, Firefox, Safari, and Edge.

17. [Usability] The system shall provide a responsive layout that is usable on both desktop and mobile screen sizes.

18. [Maintainability] The system shall keep all tunable values (multiplier sizes, chemistry bonuses, atrophy rates, roster sizes) in configuration rather than hard-coded, so they can be adjusted without code changes. Rationale: balancing these values is a stated risk.

19. [Maintainability] The system shall separate the frontend (TypeScript/React) from the backend (Python) through a documented REST/JSON API.

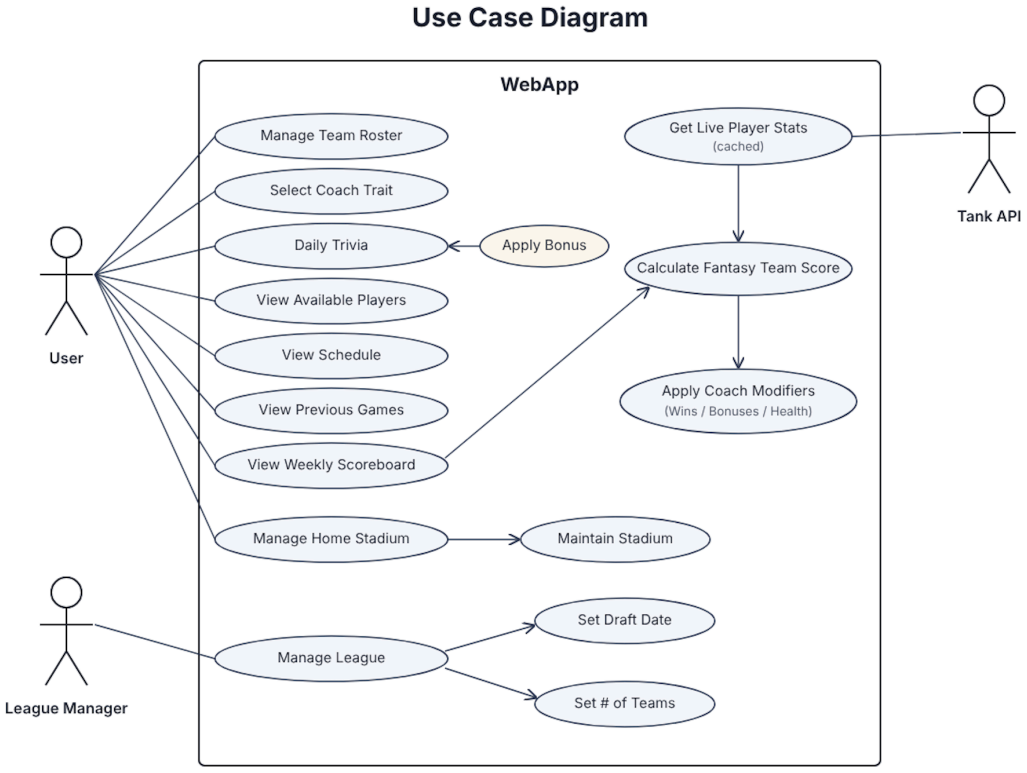
20. [Maintainability] The system shall be version-controlled and have automated tests covering the scoring calculation logic.

21. [Scalability] The system shall use a database schema and cache design that can be extended to more leagues and a paid API tier without redesign.

22. [Data Integrity] The system shall process simultaneous roster transactions (e.g., two teams claiming the same free agent) so that only one succeeds and the other is notified.

23. [Privacy] The system shall collect only the personal data needed for accounts (email, username) and not share it with third parties.

4. Use Case Diagram (10 points)



5. Class Diagram and/or Sequence Diagrams (15 points)

Increment 2 establishes the application structure for viewing weekly matchups, answering football trivia and purchasing stadium upgrades. React and TypeScript

provide the interface, Flask exposes JSON endpoints, and SQLAlchemy maps the persistent data to PostgreSQL. The following class model and three sequence diagrams describe this increment's implementation.

5.1 Architecture and changes from Increment 1

Presentation layer. The React application contains My Team, Schedule, Trivia and Stadium views, together with league and coach screens. The API client sends requests under /api/v1. During development, Vite forwards /api requests to the Flask server.

Application and data layers. The Flask application factory registers matchup, teams, trivia and admin Blueprints. Route functions perform queries and updates through Flask-SQLAlchemy. The eight database classes are shown in the two views of the class diagram in Section 5.2. Database-backed features use PostgreSQL; league membership, role and coach selections currently use browser localStorage.

Refinement of the earlier design. The previous design proposed live Tank01 statistics, Redis caching, scheduled synchronization, chemistry calculations and a weekly trivia multiplier. This increment reads stored matchup scores and awards trivia upgrade points. Stadium and training effects are represented by Stadium fields; a separate TrainingFacility class is not implemented.

Remaining design scope. Full account authentication, persistent multi-user league management, roster transactions, external statistics ingestion and automatic scoring remain future work. The original User, League, Roster, Coach, ChemistryLink and statistics-service concepts remain design targets, rather than implemented PostgreSQL classes. Browser components and Flask route functions are not additional domain model classes.

Source: application factory; React views; API client

5.2 Class diagram — core models

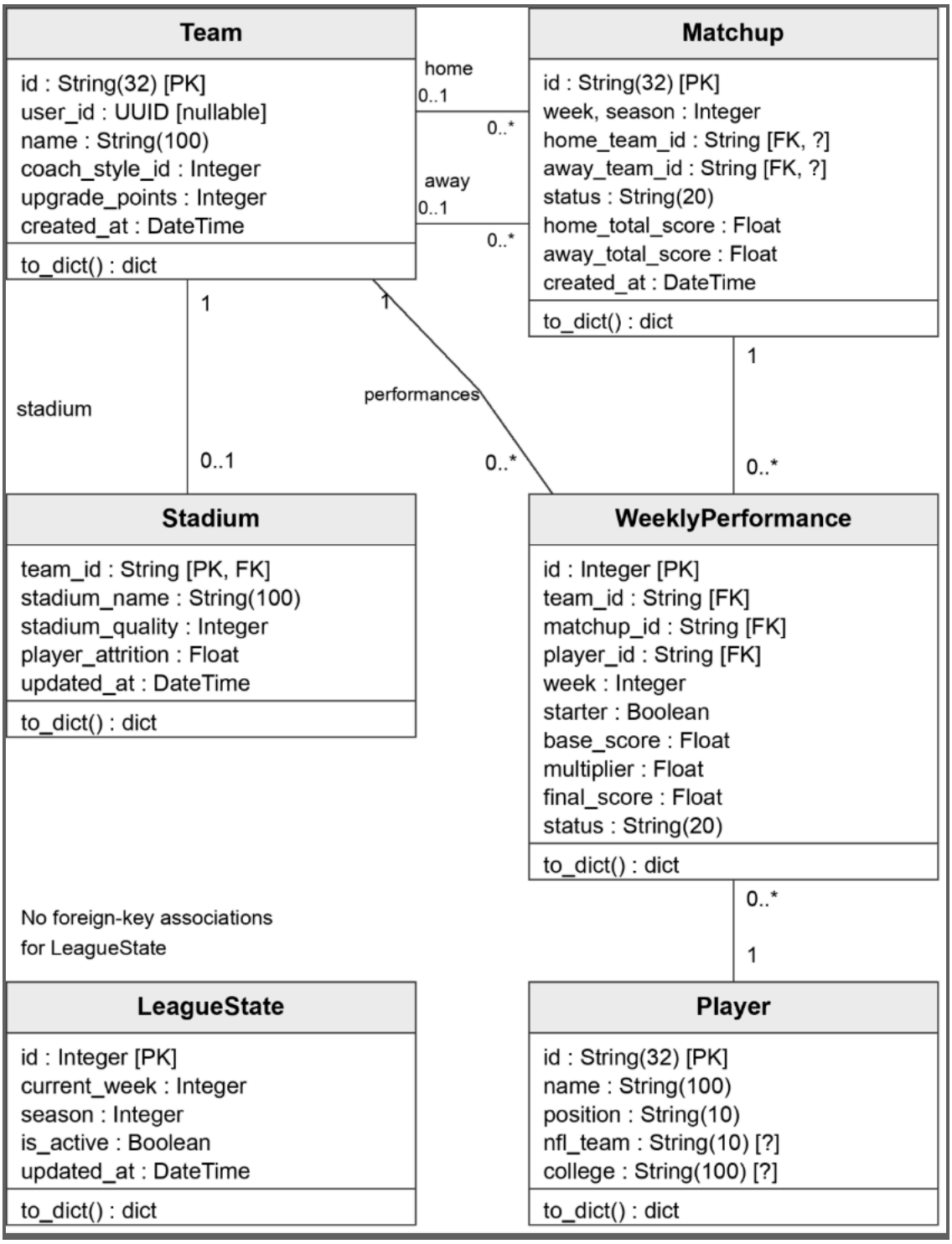


Figure 5.1. Core model view. PK = primary key; FK = foreign key; ? = nullable. Multiplicities follow the current schema. Trivia classes complete this model in Figure 5.2.

Source: SQLAlchemy model definitions

5.2 Class diagram — trivia models and relationships

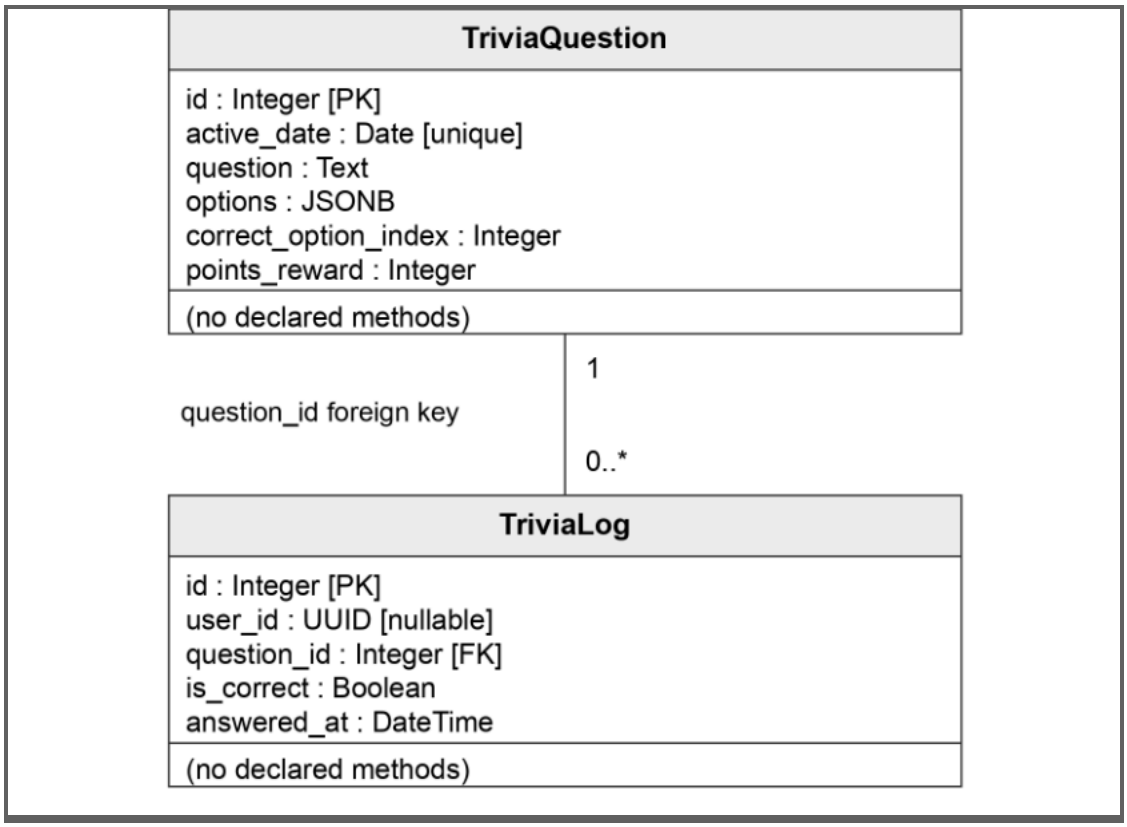


Figure 5.2. Trivia model view. A question can have many answer logs, and each log references one question. Both classes inherit SQLAlchemy model behavior and declare no custom methods.

Identity and rewards. Team.user_id and TriviaLog.user_id store user UUIDs, but neither references a User table in this increment. The submit handler uses the UUID to find the first matching Team and add the question's reward after a correct answer. This is application logic, not a foreign-key relationship between Team and TriviaLog.

Integrity and cardinality. TriviaQuestion.active_date is unique. TriviaLog has a unique constraint on (user_id, question_id). This limits a user to one attempt per question; it does not enforce a separate calendar-day limit. Team has at most one Stadium. WeeklyPerformance links one Team, one Matchup and one Player. A Matchup can reference one home and one away team, although both foreign keys currently allow null values.

Model responsibilities. The six core classes expose to_dict() methods for JSON serialization. Trivia response bodies are constructed by get_daily_question() and submit_trivia_answer(). LeagueState supplies the active week but has no foreign-key association with a persistent League model.

Source: model definitions; trivia handlers

5.3 Sequence diagram — view the current matchup

The My Team view requests the current matchup and displays the stored team totals and player performance rows. Database access in this and the following diagrams passes through SQLAlchemy.

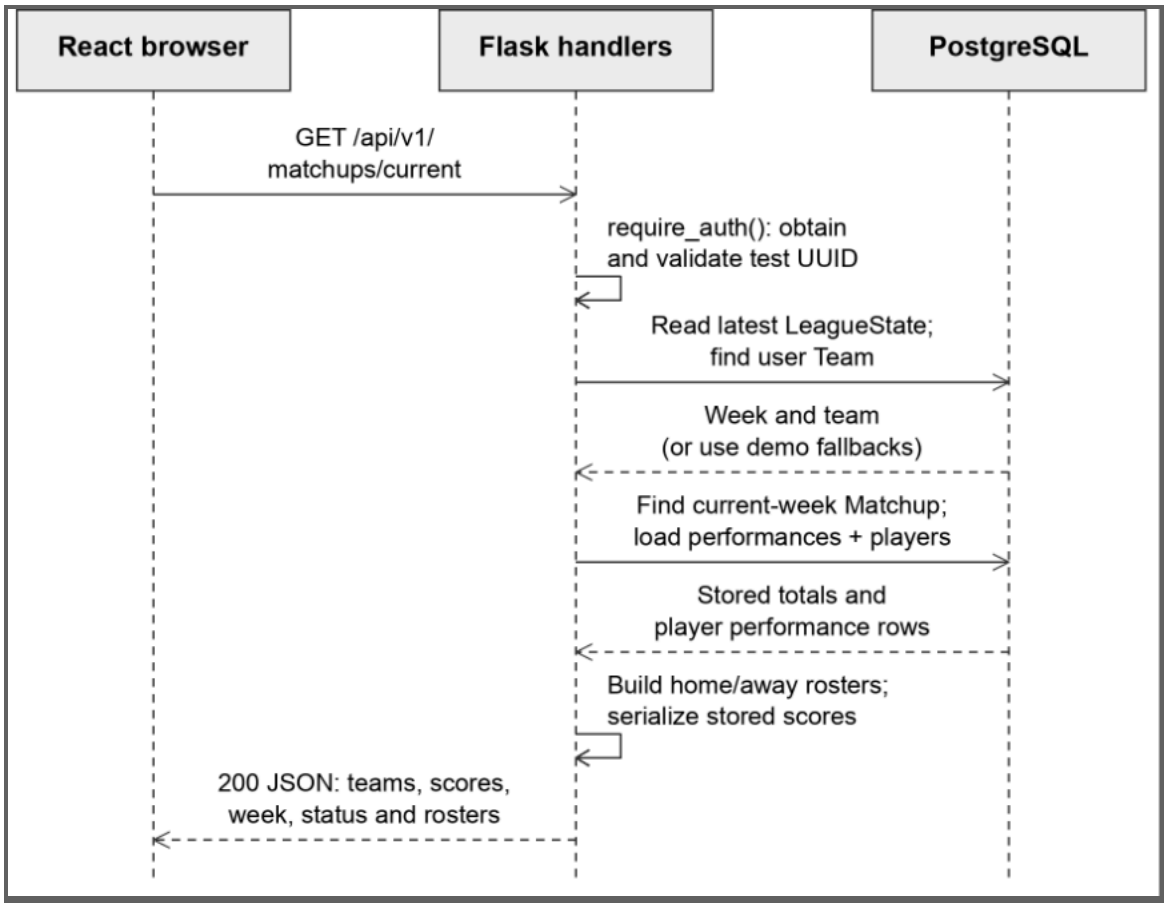


Figure 5.3. Current matchup retrieval through the React client, Flask and PostgreSQL.

Selection and fallbacks. The latest LeagueState record determines the week; week 8 is the fallback. The route finds the team linked to the test user, falling back to team_001 if necessary. It then finds a matchup for that week and separates its performance rows into home and away rosters.

Alternative outcomes. A missing team or matchup produces HTTP 404. Authentication failures return HTTP 401, or HTTP 400 for a malformed user UUID. This sequence reads existing scores; it does not calculate points or call Tank01.

Source: matchup handlers; authentication helper

5.4 Sequence diagram — answer a trivia question

The Trivia view retrieves a question and submits the selected option. The answer key stays on the server until the submission response.

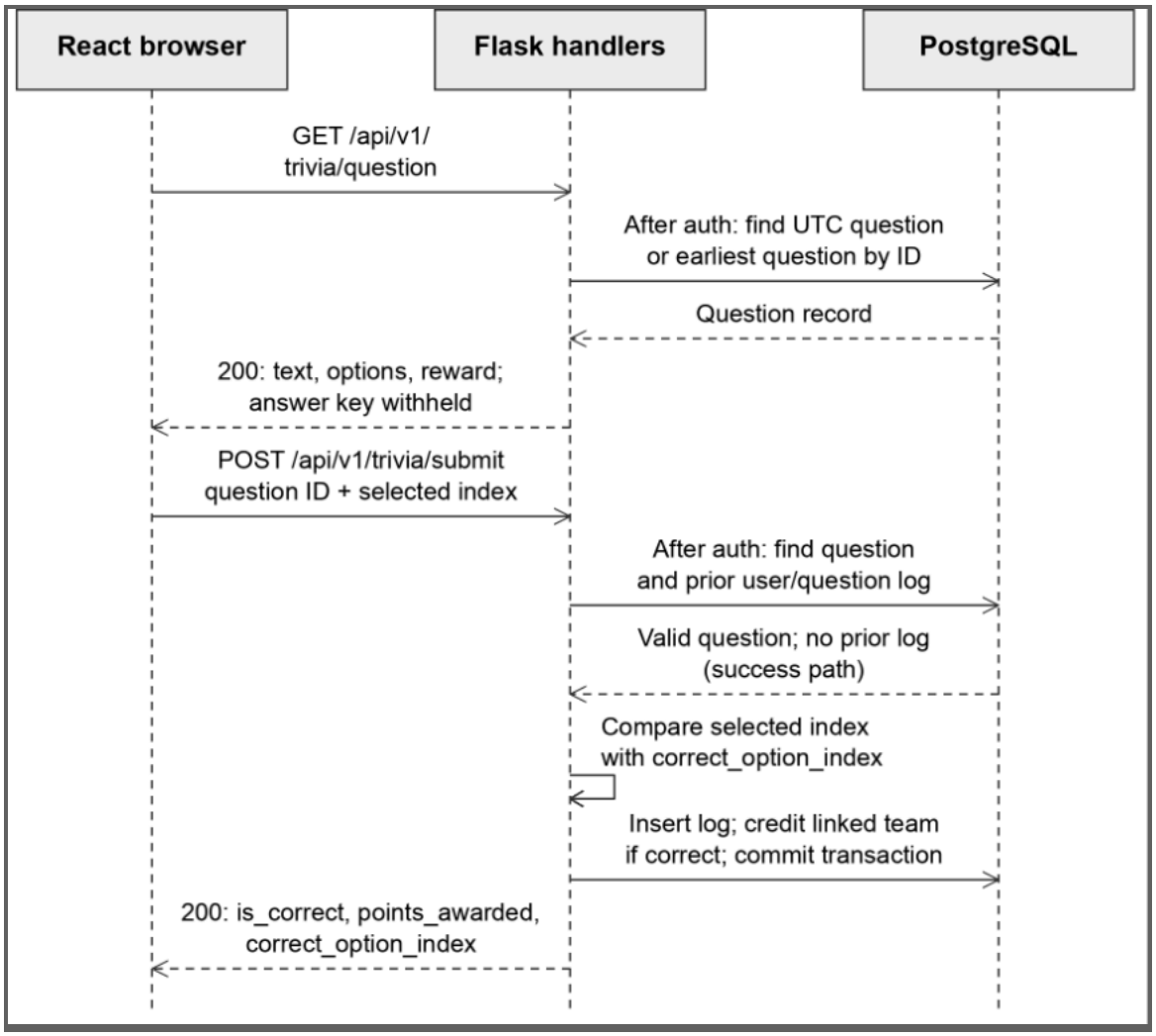


Figure 5.4. Successful trivia submission; the identity helper runs on both requests.

Reward and persistence. A correct answer awards the question's points_reward, which defaults to 25. The handler saves a TriviaLog and credits a matching Team if one exists. The response includes correctness, points awarded and the correct option index; React then shows feedback and disables the options.

Alternative outcomes. No available question or an invalid question ID returns 404. A repeated user/question attempt returns 400. The daily lookup uses UTC and falls back to the earliest question by ID. A different, unanswered question must be available for a later attempt; a weekly score multiplier and reset are not implemented.

Source: trivia handlers; Trivia view

5.5 Sequence diagram — upgrade stadium facilities

The Stadium view reads a stadium and lets the user spend upgrade points on a medical facility improvement.

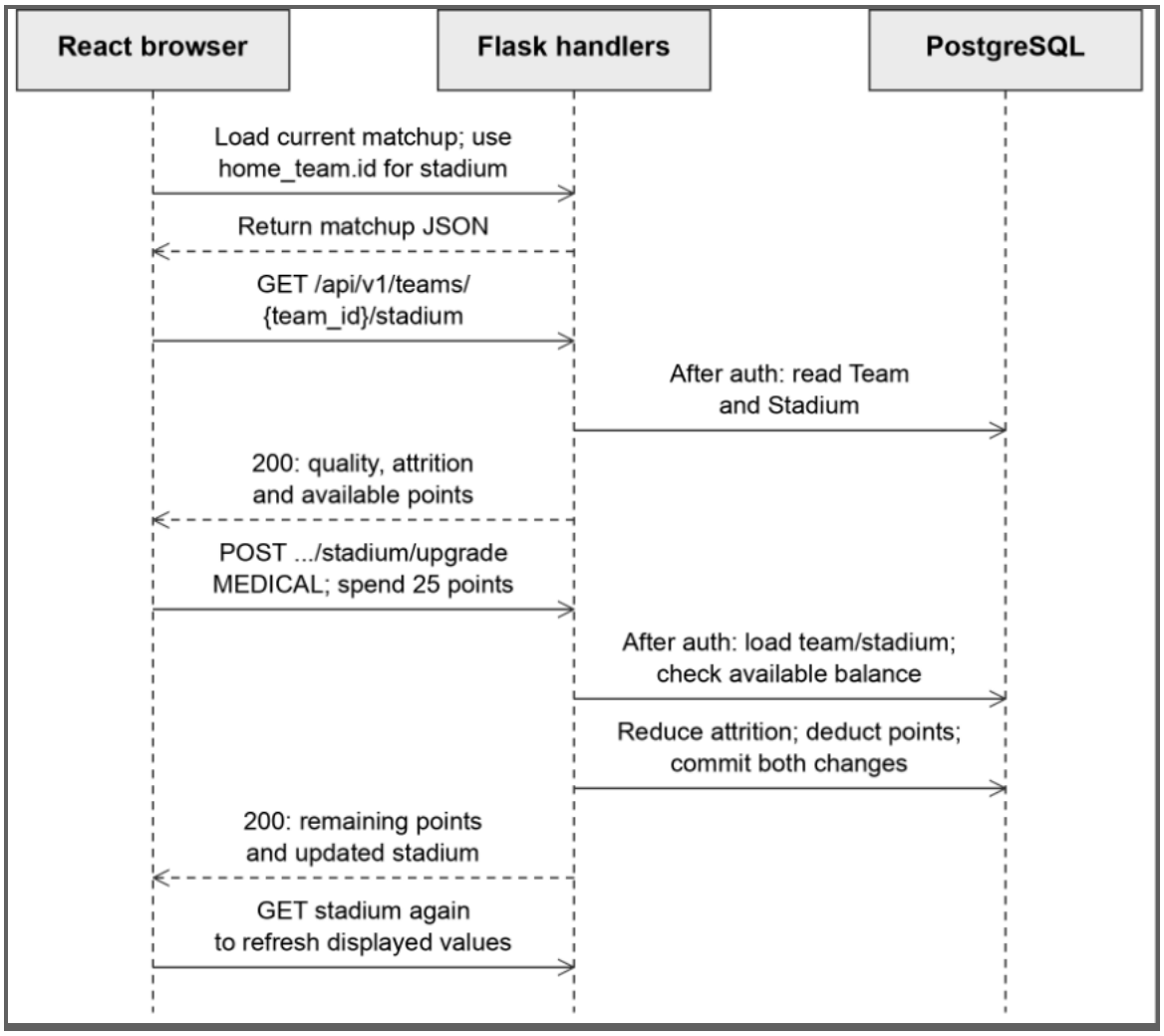


Figure 5.5. Medical upgrade from the Stadium view. The final GET returns the refreshed stadium values.

Upgrade rules. The interface sends MEDICAL with a cost of 25 points. The backend reduces player_attrition by 0.02, with a minimum of 0.05, and deducts the cost. FACILITIES has the same effect; TURF instead adds 5 to stadium_quality, capped at 100. These updates do not yet recalculate matchup scores.

Alternative outcomes and scope. Missing team or stadium records return 404; insufficient points or an unknown upgrade type returns 400. The interface currently chooses the matchup's home team. Ownership checks and strict validation of the submitted cost remain required before multi-user use.

Source: team and stadium handlers; Stadium view

6. Operating Environment (5 points)

Ultimate Fantasy Football is a browser-based application. The Increment 2 configuration supports local development with a React frontend, a Flask backend

and a reachable PostgreSQL database. A production hosting platform is not yet specified.

6.1 Client and development platform

Users need a current Chrome, Firefox, Safari or Edge browser on a desktop, laptop, tablet or phone. Cross-browser and mobile support remain compatibility targets. Development targets Windows, macOS or Linux; exact OS versions and measured hardware minimums are not established.

6.2 Software components

Component	Declared version or requirement	Role
React / TypeScript	^18.3.1 / ^7.0.2	Interface and frontend language
Vite / Node.js	^8.3.6 / ^20.19.0 or >=22.12.0	Build tools; Node range from lockfile
Python / Flask	Python not pinned / Flask 3.0.3	Backend runtime and routes
Flask-SQLAlchemy / SQLAlchemy	3.1.1 / 2.1.8	Database mapping and queries
PostgreSQL / psycopg2-binary	Server not pinned / driver 2.9.13	Storage with UUID and JSONB fields
Flask-Cors / python-dotenv	4.0.1 / 1.0.1	CORS and environment settings

6.3 Network and hosting configuration

Flask binds to 127.0.0.1 on port 5000 unless PORT is set. Vite forwards /api to that port. DATABASE_URL supplies the PostgreSQL connection; the example targets Supabase-hosted PostgreSQL. Development requires available local ports and database connectivity. Credentials belong in the local environment.

The earlier Linux hosting estimate of 1 virtual CPU and 1–2 GB RAM remains a proposed starting point, not a measured capacity result. Production service setup, HTTPS, backups and resource sizing remain deployment tasks. Redis is part of the earlier design and is not an active dependency in this increment.

Source: backend requirements; frontend manifest; lockfile; Vite configuration; backend entry point

7. Assumptions and Dependencies (5 points)

7.1 Assumptions

A1 — Prepared demonstration data. PostgreSQL tables and demo records must be provisioned: teams, stadiums, players, matchups, performances and questions. LeagueState should identify the intended week. The seed script is empty; startup does not create or populate tables.

A2 — Consistent test identity. Demonstrations set ALLOW_AUTH_BYPASS=true with a valid UUID matching Team.user_id. Without a linked team, a correct answer can report awarded points without actually crediting a team.

A3 — Daily question availability. Questions use UTC active dates in the project database; there is no external trivia service. Repeat prevention is per user/question. Full calendar-day enforcement and handling missing daily questions require refinement.

A4 — Limited demonstration scope. Browser localStorage supports leagues, roles and coach choices on one browser. Multi-user leagues need shared persistence and authorization. Sample schedules and stored scores allow demonstrations without live NFL games.

A5 — Future data and gameplay rules. Tank01 access terms, availability, rate limits and stat freshness require confirmation. The earlier 1,000-call monthly budget remains a planning assumption. Scheduled refreshes need background-job support. Chemistry data, scoring balance and weekly reset rules require validation.

7.2 Dependencies and completion conditions

The application depends on the software in Section 6 and a configured database. Dependency compatibility and installation/build checks remain necessary: the recorded React plugin peer range excludes the declared Vite major version.

Supabase token verification is planned: the current authentication helper only supports the development bypass, and the sign-in screen does not validate credentials. Server-side ownership and role checks, stricter input validation and automated tests are completion conditions for broader use. The scoring engine and test files remain placeholders.

Trivia upgrade points replace the earlier proposed trivia practice multiplier in the current implementation. Applying a weekly trivia multiplier, resetting it, importing Tank01 statistics and operating a Redis cache remain deferred requirements. Those features depend on future service, scheduling and persistence work.

Source: auth helper; environment example; local browser state; trivia behavior